

SOM-Seq: A Python Toolbox for Single-Cell Sequencing Simulation and Self-Organizing Map Analysis

Zachary Caterer ^{1,2,3*}, Madeline Pernet ^{4*}, and Victoria Hurd ^{5*}

¹ Department of Chemical and Biological Engineering, University of Colorado Boulder, Boulder, CO, USA ² Department of Biomedical Informatics, University of Colorado Anschutz Medical Campus, Aurora, CO, USA ³ Biofrontiers Institute Interdisciplinary Biology PhD Program, University of Colorado Boulder, Boulder, CO, USA ⁴ Department of Civil, Environmental, and Architectural Engineering, University of Colorado Boulder, Boulder, CO, USA ⁵ Department of Aerospace Engineering Sciences, University of Colorado Boulder, Boulder, CO, USA  Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Single-cell sequencing technologies profile gene expression in individual cells, enabling researchers to identify distinct cell populations and understand how they change in disease. Analyzing these datasets requires dimensionality reduction and clustering methods to reveal biologically meaningful structure. Self-Organizing Maps (SOMs) are a class of unsupervised neural network that project high-dimensional data onto a two-dimensional grid of neurons, where neighboring neurons represent similar regions of the input space, explicitly encoding topological relationships between clusters that standard methods such as UMAP or t-SNE do not preserve. Despite this interpretive advantage, SOMs remain underutilized in the single-cell community, in part because no existing tool combines SOM-based analysis with single-cell-style simulation in a single package. Researchers wishing to benchmark SOM-based clustering must currently assemble a simulation tool and a separate SOM library themselves.

SOM-Seq is an open-source Python toolbox that addresses this gap by integrating two complementary workflows: simulated single-cell dataset generation (Seq_Sim) and SOM-based clustering and visualization (SOM). The Seq_Sim module generates realistic simulated single-cell datasets — producing per-cell feature matrices with configurable cell-type compositions, batch effects, disease states, and differential expression patterns — providing a reproducible, ground-truth-labeled environment for benchmarking clustering methods without requiring access to patient data. This simulation approach is adapted from methods developed in the Zhang Lab (Inamo & Young, 2024). The SOM module provides a high-level Python class built on MiniSom (Vettigli, 2018) that handles data scaling, automated hyperparameter tuning, topographic quality metrics, and publication-quality visualizations including component planes and categorical overlays. Both modules expose command-line interfaces (CLIs), making them composable within broader bioinformatics pipelines or usable as standalone tools.

Statement of Need

Self-Organizing Maps (SOMs), introduced by Kohonen (Kohonen, 1990), produce a discrete two-dimensional grid of neurons in which neighboring neurons represent similar regions of the input feature space, explicitly encoding topological structure between clusters. Despite this interpretive advantage, SOMs remain underutilized in the single-cell community. A key barrier is the absence of a well-tested, end-to-end Python package that combines SOM fitting with

41 single-cell-style simulation, reducing the friction required to benchmark SOM-based clustering
42 against other methods on controlled synthetic data.

43 SOM-Seq addresses this gap in two ways. First, Seq_Sim generates synthetic datasets that
44 statistically mirror real single-cell data—including heterogeneous cell-type proportions, batch
45 variability, and disease-associated fold changes—providing researchers with a reproducible,
46 ground-truth-labeled environment for method comparison without requiring access to patient
47 data. Second, the SOM module wraps the complete SOM workflow (scaling, training, metric
48 evaluation, and visualization) into a clean Python API and CLI, lowering the expertise required
49 to apply SOM-based analysis to tabular omics data.

50 Together, these modules enable researchers to simulate a dataset with known structure, fit a
51 SOM, and immediately evaluate clustering quality using Percent Variance Explained (PVE)
52 and topographic error—a capability not available in existing single-cell analysis frameworks.

53 State of the Field

54 High-dimensional single-cell RNA sequencing (scRNA-seq) data requires dimensionality
55 reduction and clustering to reveal biologically meaningful structure (Luecken & Theis, 2019).
56 Established tools such as Seurat (Hao et al., 2021) and Scanpy (Wolf et al., 2018) are the
57 standard for single-cell analysis, typically pairing graph-based community detection with t-SNE
58 (Maaten & Hinton, 2008) or UMAP (McInnes et al., 2018) for visualization. While powerful,
59 these methods embed data into a continuous low-dimensional space that does not explicitly
60 preserve the topological distances between clusters, making it difficult to reason about the
61 relative proximity (i.e., similarity) of cell populations.

62 At the algorithmic level, general-purpose SOM libraries such as MiniSom (Vettigli, 2018) expose
63 raw training routines but provide no single-cell-specific workflow: users must independently
64 implement data scaling, select grid dimensions and neighborhood parameters, compute quality
65 metrics such as topographic error and percent variance explained (PVE), and build visualizations
66 suited to omics data. Similarly, single-cell simulators such as the SCORPIO framework (Inamo
67 & Young, 2024), on which Seq_Sim is based, are not paired with an SOM analysis workflow.
68 While a motivated user could combine these tools manually, doing so requires non-trivial
69 implementation effort: writing a hyperparameter search over grid dimensions and neighborhood
70 functions, implementing the PVE and topographic error calculations, and building component
71 plane and categorical overlay visualizations — none of which MiniSom or SCORPIO provide.
72 SOM-Seq packages all of these steps behind a single, consistent API and CLI, reducing the
73 expertise and implementation effort required to apply SOM-based analysis to single-cell data.
74 To our knowledge, no existing Python package integrates this complete SOM workflow, scaling,
75 tuning, quality metrics, and omics-specific visualization, with single-cell-style data simulation
76 in one tool.

77 Software Design

78 SOM-Seq's design reflects two deliberate trade-offs. First, rather than requiring real patient-
79 derived single-cell data, Seq_Sim generates statistically realistic synthetic data with known
80 ground-truth cell-type labels; this sacrifices biological realism for reproducibility and removes
81 the privacy and data-access barriers that complicate benchmarking clustering methods on
82 human subject data. Second, rather than reimplementing SOM training, the SOM module
83 wraps the existing, well-tested MiniSom library (Vettigli, 2018) and adds the scaling, tuning,
84 metric, and visualization steps that a complete single-cell workflow requires; this keeps the
85 core training algorithm maintained upstream while concentrating SOM-Seq's engineering effort
86 on the parts of the workflow specific to omics data analysis.

Sequence Simulation (Seq_Sim)

The Seq_Sim module generates synthetic single-cell datasets by constructing a subject-level metadata table (age, sex, disease status, batch) and a cell-type composition matrix. Cell counts for major and rare cell populations are drawn from uniform distributions parameterized by user-supplied standard deviations and relative abundances. Disease-associated differential abundance is introduced by adding or removing cells of specified types in proportion to a configurable fold-change parameter. Pseudo-feature expression matrices are then generated per cell by combining cluster-specific signal, disease variance, and individual-level variance, with additive Gaussian noise controlled by a cluster ratio parameter. All random operations accept a seed argument to guarantee reproducibility.

Key configurable parameters exposed via config.yml or CLI include:

- num_samples: number of subjects to simulate
- fold_change: magnitude of disease-associated differential abundance
- n_major_cell_types, n_minor_cell_types: cell-type composition
- n_features: number of pseudo-expression features per cell
- n_batches, prop_disease, prop_sex: study design parameters

Self-Organizing Maps (SOM)

The SOM module wraps MiniSom (Vettigli, 2018) into a SOM Python class that manages the full analysis workflow:

1. **Input validation:** type and range checks on all constructor arguments.
2. **Scaling:** z-score or min-max normalization with corresponding inverse transforms stored for weight unscaling.
3. **Training:** configurable grid dimensions, rectangular or hexagonal topology, and Gaussian or bubble neighborhood functions.
4. **Metrics:** PVE measures how much of the input variance is captured by the neuron weight vectors; topographic error (Kohonen, 1990) quantifies how often a data point's two best-matching units are non-adjacent on the grid.
5. **Hyperparameter tuning:** when multiple values are supplied for any hyperparameter, the CLI evaluates all combinations and selects the configuration maximizing $PVE - 100 \times \text{topographic_error}$.
6. **Visualization:** component plane plots (one heatmap per input feature) and categorical overlay plots (one heatmap per categorical variable), saved as publication-quality PNG files.

Usage

Generating Sequencing Data

```
python Seq_Sim/seq_sim.py --num_samples 30 --fold_change 0.5 --config_file Seq_Sim/confi
```

Fitting a SOM

```
python SOM/som.py -t data/sim_data_pseudo_feature_num_samples_30_fc_0.5.csv -c data/sim
```

Python API

```
import pandas as pd
from SOM.utils.som_utils import SOM

train = pd.read_csv("data/sim_data_pseudo_feature_num_samples_30_fc_0.5.csv")
meta = pd.read_csv("data/sim_data_latent_data_num_samples_30_fc_0.5.csv")
```

```
som = SOM(
    train_dat=train,
    other_dat=meta,
    scale_method="zscore",
    x_dim=5,
    y_dim=4,
    topology="hexagonal",
    neighborhood_fnc="gaussian",
    epochs=100,
)
som.train_map()
print(f"PVE: {som.calculate_percent_variance_explained():.1f}%")
print(f"Topographic error: {som.calculate_topographic_error():.3f}")
som.plot_component_planes(output_dir="output/")
som.plot_categorical_data(output_dir="output/")
```

Testing and Documentation

SOM-Seq ships with a pytest test suite covering both modules, including input validation, scaling round-trips, training, metric calculations, and plot generation. Continuous integration via GitHub Actions runs the full suite on each push. API documentation is hosted on Read the Docs.

Research Impact Statement

Beyond its origin as a course project, the SOM module is actively being applied in ongoing biomedical informatics research as a topology-preserving alternative to graph-based and explainable ML clustering pipelines. Specifically, it is being evaluated on publicly available single-cell datasets — COVID-19 PBMC multi-omics data (Stephenson et al., 2021) and ulcerative colitis tissue transcriptomics (Smillie et al., 2019) — as a complementary benchmarking tool alongside CellPhenoX (Young et al., 2025), a published explainable machine learning method for single-cell clinical phenotyping. The research question being addressed is whether topology-preserving SOM-based clustering identifies biologically coherent cell populations consistent with those identified by CellPhenoX on the same datasets. Reproducible example analyses are provided in the repository: Seq_Sim/walkthrough/seq_sim_workflow.ipynb demonstrates the full data simulation workflow, and SOM/examples/ contains three worked examples applying the SOM module to the Iris dataset (som_example_iris.ipynb), the Titanic dataset (som_example_titanic.ipynb), and simulated single-cell data (som_example_seq.ipynb). Community-readiness is further supported by a pytest test suite exercising both modules, continuous integration on every push, published API documentation, an OSI-approved open-source license, and packaging for installation via PyPI.

Acknowledgements

This work originated as part of the University of Colorado Boulder course **CSCI 6118: Software Engineering for Scientists**, created by **Dr. Ryan Layer** and taught by **Dr. Erik Johnson**. We thank both Dr. Layer and Dr. Johnson for their guidance and for creating the collaborative environment that made this project possible. This work was also supported in part by the U.S. National Science Foundation under **Award No. 2022138**.

AI usage disclosure

We used Anthropic's Claude (Sonnet 4.6) to assist with refining documentation and addressing reviewer comments, including adding clarity, removing redundant information, and drafting revised paper sections. We also used Anthropic's Claude (Sonnet 5, via Claude Code) during pre-submission preparation to identify and fix two command-line interface bugs (an argument-parsing crash in the SOM CLI and an invalid function call in the Seq_Sim CLI), to add continuous integration workflows for package build validation and PyPI publishing, and to correct inconsistencies in the LICENSE and README. During revision, Claude (Sonnet 4.6) assisted in identifying and fixing four additional bugs: an incorrect topographic error calculation for hexagonal grids, a broken fold-change effect in the simulation module, missing support for optional metadata in the CLI and API, and a non-functional `-m` plot suppression flag. All AI-assisted content and code changes were reviewed by the authors, who validated correctness and made all core design decisions.

References

- Hao, Y., Hao, S., Andersen-Nissen, E., Mauck, W. M., Zheng, S., Butler, A., Lee, M. J., Wilk, A. J., Darby, C., Zager, M., & others. (2021). Integrated analysis of multimodal single-cell data. *Cell*, 184(13), 3573–3587. <https://doi.org/10.1016/j.cell.2021.04.048>
- Inamo, J., & Young, J. (2024). *SCORPIO: Simulated single-cell omics with realistic phenotypes and integrated outcomes*. <https://github.com/fanzhanglab/SCORPIO>
- Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE*, 78(9), 1464–1480. <https://doi.org/10.1109/5.58325>
- Luecken, M. D., & Theis, F. J. (2019). Current best practices in single-cell RNA-seq analysis: A tutorial. *Molecular Systems Biology*, 15(6), e8746. <https://doi.org/10.15252/msb.20188746>
- Maaten, L. van der, & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605. <https://jmlr.org/papers/v9/vandermaaten08a.html>
- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv Preprint arXiv:1802.03426*. <https://doi.org/10.48550/arXiv.1802.03426>
- Smillie, C. S., Biton, M., Ordovas-Montanes, J., Sullivan, K. M., Burgin, G., Graham, D. B., Herbst, R. H., Rogel, N., Slyper, M., Waldman, J., & others. (2019). Intra- and inter-cellular rewiring of the human colon during ulcerative colitis. *Cell*, 178(3), 714–730. <https://doi.org/10.1016/j.cell.2019.06.029>
- Stephenson, E., Reynolds, G., Botting, R. A., Calero-Nieto, F. J., Morgan, M. D., Tuong, Z. K., Bach, K., Sungnak, W., Worlock, K. B., Yoshida, M., & others. (2021). Single-cell multi-omics analysis of the immune response in COVID-19. *Nature Medicine*, 27(5), 904–916. <https://doi.org/10.1038/s41591-021-01329-2>
- Vettigli, G. (2018). *MiniSom: Minimalistic and NumPy-based implementation of the self organizing map*. <https://github.com/JustGlowing/minisom>
- Wolf, F. A., Angerer, P., & Theis, F. J. (2018). SCANPY: Large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1), 15. <https://doi.org/10.1186/s13059-017-1382-0>
- Young, J., Inamo, J., Caterer, Z., Krishna, R., & Zhang, F. (2025). CellPhenoX: An explainable machine learning method for identifying cell phenotypes to predict clinical outcomes from single-cell multi-omics. *Advanced Science*, 12, e03289. <https://doi.org/10.1002/adv.202503289>